

SIMATS
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

CASE STUDY REPORT

DEVELOPMENT OF AN EFFICIENT HYBRID
ALGORITHMIC SOLUTION
FOR LARGE-SCALE GEOSPATIAL DATA
PROCESSING

Submitted by

BANDI MOHITH MANOJ

Register No.: 192472018

KODANDA RAM T

Register No.: 192524162

M YASODHA KRISHNA

Register No.: 192472100

Faculty: Dr. F. Mary Harin Fernandez

Academic Year: 2026–2027

Submission Date: 1 September 2026

ABSTRACT

This case study develops and evaluates an adaptive hybrid solution to the Closest-Pair-of-Points problem for large-scale geospatial coordinate data. The study uses the OpenFlights airport database as a real-world coordinate dataset and compares three algorithmic strategies: a Brute-Force method, a Divide-and-Conquer method, and a developed Hybrid method that switches to Brute Force when a recursive subproblem becomes smaller than an experimentally selected threshold. The experimental workflow validates correctness, measures execution time, counts distance calculations, measures traced peak memory allocation, and evaluates scalability across increasing input sizes. The robust threshold experiment evaluates candidate thresholds using repeated timing across multiple problem sizes and selects $k = 8$ for the final benchmark. At the largest tested input size of $n = 7500$, Brute Force required a median execution time of 3.242633 s and 28,121,250 distance calculations, while Divide and Conquer required 0.032881 s and 9,002 distance calculations. The Hybrid algorithm required 0.025702 s and 24,887 distance calculations while returning the same closest pair as the other approaches. Thus, the Hybrid method was 21.83% faster than the pure Divide-and-Conquer implementation and 99.21% faster than Brute Force at $n = 7500$ in this implementation and environment. The results demonstrate how algorithmic hybridization can improve practical performance while preserving the asymptotic scalability of the divide-and-conquer strategy.

Keywords: Closest Pair, Brute Force, Divide and Conquer, Hybrid Algorithm, Geospatial Data, Threshold Optimization, Algorithm Analysis, Scalability

TABLE OF CONTENTS

1. Introduction
 2. Problem Understanding and Formulation
 3. Dataset Description
 4. Team Contribution Classification
 5. Application of Course Knowledge
 6. Theoretical Analysis
 7. Solution Design and Methodology
 8. Implementation and Modern Tool Usage
 9. Experimental Methodology
 10. Results and Validation
 11. Analysis and Engineering Discussion
 12. Engineering Decision
 13. Broader Considerations and SDG 9
 14. Limitations
 15. Future Optimization
 16. Conclusion
 17. Individual Student Reflections
 18. References
 19. Assignment-Requirement Coverage Matrix
- Appendix A – Final Experimental Summary
- Appendix B – Repository Structure
- Appendix C – AI-Assisted Tool Use

LIST OF FIGURES

Figure 1. Overall architecture and hybrid algorithm workflow

Figure 2. Median execution time on a logarithmic scale versus input size

Figure 3. Distance computations versus input size

Figure 4. Peak memory versus input size

Figure 5. Robust hybrid threshold experiment

Figure 6. Median execution time versus input size (linear scale)

Figure 7. Representative source-code evidence: Brute Force implementation

Figure 8. Representative source-code evidence: Divide-and-Conquer implementation

Figure 9. Representative source-code evidence: dataset preparation

Figure 10. Representative source-code evidence: threshold-search configuration

LIST OF TABLES

Table 1. Student contribution classification

Table 2. Dataset fields relevant to the case study

Table 3. Algorithm comparison before experimentation

Table 4. Experimental configuration

Table 5. Final performance at the largest benchmark size

Table 6. Derived performance indicators at $n = 7500$

Table 7. Assignment-requirement coverage matrix

1. INTRODUCTION

1.1 Case Background

Large-scale infrastructure and location-planning systems frequently operate on collections of geographic coordinates. Examples include airports, weather stations, service facilities, cell towers, logistics hubs, and other spatial assets. A recurring computational question is whether two facilities are unusually close to one another. Finding the closest pair of points provides a simple but useful primitive for detecting redundancy, possible operational conflicts, or opportunities for service consolidation.

The assignment brief identifies the Closest-Pair-of-Points problem as an appropriate case for studying algorithmic efficiency because a direct pairwise comparison strategy becomes expensive as the number of locations grows, whereas a divide-and-conquer strategy reduces the number of distance evaluations. The assignment further requires a meaningful hybridization, experimental threshold selection, and quantitative comparison of all approaches. [1]

1.2 Aim

The aim of this project is to design, implement, optimize, and evaluate a Hybrid Closest-Pair algorithm that combines the correctness and simplicity of Brute Force for small subproblems with the scalability of Divide and Conquer for larger subproblems.

1.3 Objectives

- Prepare a real-world geospatial coordinate dataset from OpenFlights.
- Implement a Brute-Force Closest-Pair algorithm.
- Implement a Divide-and-Conquer Closest-Pair algorithm using median x-coordinate partitioning and strip processing.
- Develop a Hybrid algorithm that switches to Brute Force below a configurable threshold k .
- Experimentally determine a robust threshold using repeated measurements across multiple input sizes.
- Compare execution time, distance calculations, traced peak memory allocation, correctness, and scalability.
- Use the results to make an evidence-based engineering decision for large-scale geospatial processing.

1.4 Research Question

Can a threshold-based Hybrid Closest-Pair algorithm reduce practical execution time relative to a pure Divide-and-Conquer implementation while maintaining correctness and large-input scalability on real-world geospatial coordinate data?

1.5 Scope

The scope is deliberately aligned with the course assignment. The algorithms operate on two-dimensional coordinates and use Euclidean distance as required. Geographic coordinates are treated as planar x-y values for the algorithmic experiment. The study does not attempt to compute great-circle distance or solve a broader geographic information system problem.

2. PROBLEM UNDERSTANDING AND FORMULATION

2.1 Problem Definition

Given a set P of n points in the plane, identify two distinct points p and q such that the Euclidean distance between them is minimum among all point pairs. The output consists of the two point records and the corresponding minimum distance.

2.2 Input

A collection of records containing valid latitude and longitude values extracted from the OpenFlights airports dataset. The final dataset used in the experiment contained 7,698 valid coordinate records.

2.3 Output

The algorithm returns the two closest named locations and their minimum Euclidean separation in the coordinate representation used by the experiment.

2.4 Euclidean Distance Model

For points $p1 = (x1, y1)$ and $p2 = (x2, y2)$, the required distance is:

$$d(p1, p2) = \sqrt{(x1 - x2)^2 + (y1 - y2)^2}$$

The implementation compares squared distances internally to avoid unnecessary square-root operations during most candidate comparisons and takes the square root only when a result distance is required. The reported separation therefore remains the Euclidean distance in the latitude/longitude coordinate space.

2.5 Computational Challenge

For Brute Force, every possible pair must be considered. The number of pairs is $n(n-1)/2$, producing quadratic growth. As n increases from hundreds to thousands,

the number of distance computations grows rapidly, making the direct approach unsuitable for large input sizes.

2.6 Assumptions and Constraints

- Only rows containing valid numeric coordinates within latitude $[-90, 90]$ and longitude $[-180, 180]$ are retained.
- All three algorithms receive identical point subsets for fair comparison.
- Euclidean distance is used exactly as required by the assignment.
- The Hybrid algorithm switches to Brute Force when a recursive subproblem has size $\leq k$.
- Benchmark sizes are limited by practical execution cost, especially for Brute Force.
- Timing results are treated as empirical measurements specific to the tested software and hardware environment.

3. DATASET DESCRIPTION

3.1 OpenFlights Dataset

The OpenFlights airport database provides airport identifiers, names, city and country information, IATA/ICAO codes, latitude, longitude, altitude, timezone, and related attributes. OpenFlights describes the latitude field as decimal degrees and the longitude field as decimal degrees. [2]

3.2 Relevant Attributes

Field	Description	Use in this study
Airport ID	Unique OpenFlights identifier	Record identity
Name	Airport or location name	Displayed in closest-pair output
City	Main city served	Contextual metadata
Country	Country or territory	Contextual metadata
Latitude	Decimal latitude	x-coordinate used by algorithm
Longitude	Decimal longitude	y-coordinate used by algorithm
Altitude	Elevation in feet	Not used in closest-pair calculation
Timezone	UTC offset	Not used in closest-pair calculation
Type	Airport or related location type	Used only as source metadata, not in distance computation

3.3 Data Preparation

The Python loader reads the OpenFlights file using the csv module. Rows shorter than the required fields or containing invalid coordinate values are skipped. Valid latitude and longitude values are converted to floating-point values and stored as Point objects along with the location name. Representative implementation evidence is included in Section 8.3.

3.4 Dataset Size

The final run reported 7,698 valid coordinate records. The largest benchmark size was $n = 7,500$, selected to stay within practical computational limits while leaving the full dataset available for preprocessing and validation.

3.5 Dataset Suitability

The dataset is well suited to the assignment because each valid record naturally maps to a 2-D coordinate point, and the dataset is large enough to demonstrate the gap between quadratic pair enumeration and a scalable divide-and-conquer strategy. OpenFlights also provides an explicit data-download source that can be cited and reproduced. [2]

4. TEAM CONTRIBUTION CLASSIFICATION

The work is classified into complementary modules so that all three students contribute to algorithm development, experimentation, and technical communication.

Student	Primary responsibility	Secondary responsibility
BANDI MOHITH MANOJ	Hybrid algorithm development and threshold optimization	Integration, performance interpretation, final engineering decision
KODANDA RAM T	Dataset preparation and experimental benchmarking	Runtime, distance-count and memory measurement; result organization
M YASODHA KRISHNA	Theoretical analysis and correctness validation	Graphs, discussion, SDG 9 linkage and report documentation

5. APPLICATION OF COURSE KNOWLEDGE

5.1 CO1 – Algorithm Efficiency Analysis

CO1 is addressed through asymptotic analysis, non-recursive operation counting, recursive analysis, and the Master Theorem for the divide-and-conquer recurrence.

5.2 CO2 – Brute Force

CO2 is addressed by explicitly generating and testing every pair of points, measuring the exact number of distance calculations, and demonstrating quadratic growth experimentally.

5.3 CO3 – Divide and Conquer

CO3 is addressed by sorting points, recursively partitioning them around the median x-coordinate, solving the two subproblems, and checking the cross-boundary strip.

5.4 Integration into the Hybrid Approach

The Hybrid algorithm integrates CO2 and CO3 by using the Brute-Force method as a leaf-level solver inside the Divide-and-Conquer recursion. This creates a practical adaptation rather than merely reproducing the two textbook methods separately.

6. THEORETICAL ANALYSIS

6.1 Brute-Force Complexity

The Brute-Force method evaluates all unordered point pairs. The number of pair checks is:

$$C(n, 2) = n(n - 1) / 2$$

Therefore the running time is quadratic: $O(n^2)$. Because the algorithm always enumerates all pairs, the lower and tight bounds are also $\Omega(n^2)$ and $\Theta(n^2)$, respectively.

6.2 Divide-and-Conquer Complexity

After points are ordered by x and y coordinates, the recursive solver divides the point set into two approximately equal halves. Each half is solved recursively, and a linear-time strip-processing phase examines candidate cross-boundary pairs. The resulting recurrence is:

$$T(n) = 2T(n/2) + O(n)$$

6.3 Master Theorem Solution

For $T(n) = aT(n/b) + f(n)$, the parameters are $a = 2$, $b = 2$, and $f(n) = O(n)$. Hence:

$$n^{(\log_b a)} = n^{(\log_2 2)} = n$$

Since $f(n) = \Theta(n^{(\log_b a)})$, the equal-growth case of the Master Theorem applies, yielding:

$$T(n) = \Theta(n \log n)$$

6.4 Hybrid Complexity

The Hybrid algorithm retains the same divide-and-conquer framework for large subproblems, so its asymptotic behavior remains $\Theta(n \log n)$. The threshold changes the constant factors by reducing recursive overhead and replacing very small recursive instances with direct Brute-Force evaluation. The main optimization problem is therefore practical threshold selection rather than asymptotic improvement beyond $n \log n$.

6.5 Space Considerations

The implementation maintains x-sorted and y-sorted point lists and creates intermediate lists such as recursive halves and strip candidates. The measured peak allocation therefore grows with input size for Divide and Conquer and Hybrid. The Brute-Force procedure performs pair comparisons without building the same recursive auxiliary structures, which explains its much smaller additional traced allocation in the experiment. These figures represent tracemalloc-observed peak allocations during the measured call and not total operating-system process memory.

7. SOLUTION DESIGN AND METHODOLOGY

7.1 Overall Workflow

The solution follows the assignment sequence: Input Dataset → Data Preparation → Brute Force → Divide and Conquer → Hybrid Development → Threshold Determination → Complexity Analysis → Experimental Evaluation → Optimization → Performance Comparison → Final Decision. [1]

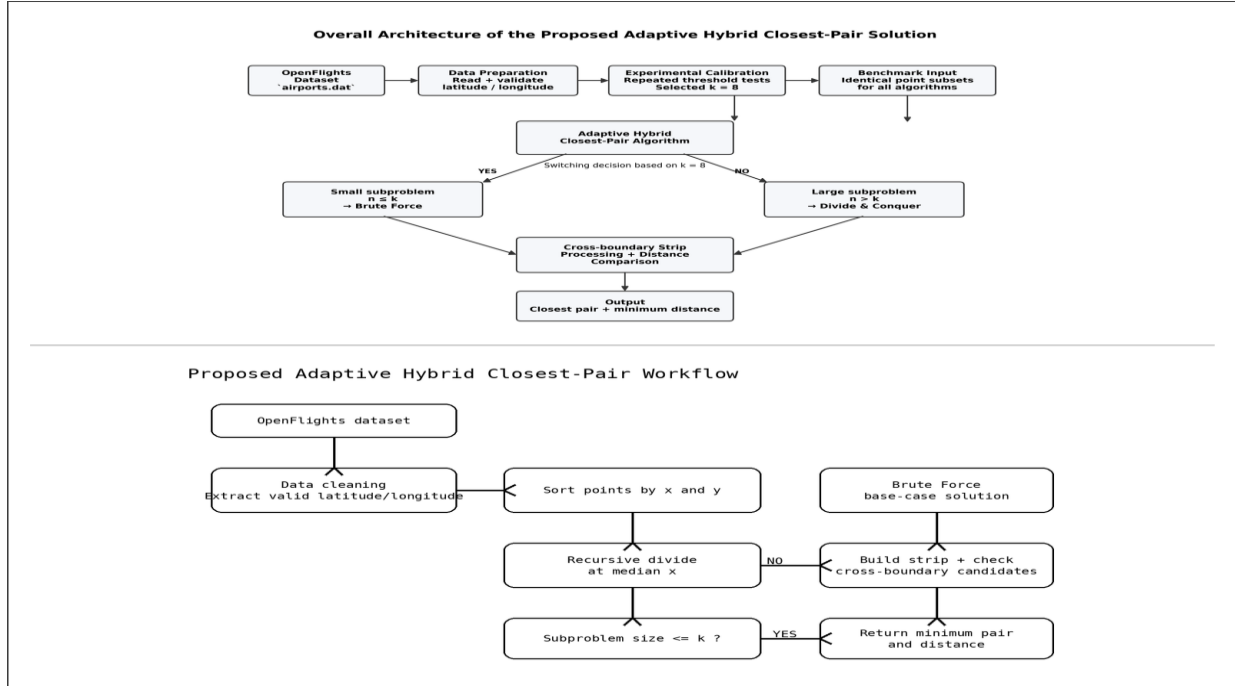


Figure 1. Overall architecture and hybrid algorithm workflow.

7.2 Brute-Force Algorithm

Pseudocode:

1. Set bestDistance to infinity.
2. For each i from 0 to $n-2$:
 - a. For each j from $i+1$ to $n-1$:
 - i. Compute the Euclidean distance.
 - ii. Replace the best pair if smaller.
3. Return the best pair and distance.

7.3 Divide-and-Conquer Algorithm

Pseudocode:

1. If the subproblem has at most three points, use Brute Force.
2. Split P_x at the median x -coordinate.
3. Partition P_y into left and right subsets.
4. Recursively solve both halves.
5. Keep the smaller of the two distances.
6. Build the strip around the split line.

7. Check valid nearby strip pairs in y-order.
8. Return the smallest pair found.

7.4 Hybrid Algorithm

Pseudocode:

1. If subproblem size $\leq k$, use Brute Force.
2. Otherwise prepare P_x and P_y .
3. Recursively solve left and right halves.
4. Determine the current best distance.
5. Build and inspect the cross-boundary strip.
6. Return the global minimum pair.

7.5 Switching Strategy

The switching rule is deterministic: when the recursive subproblem size is less than or equal to k , the solver invokes the Brute-Force routine. Otherwise it continues with the divide-and-conquer recursion. The threshold k therefore controls a trade-off between recursive overhead and local pair enumeration.

7.6 Threshold Determination Method

The improved experiment evaluates k values $\{2, 4, 8, 12, 16, 20, 24, 28, 32, 40, 48, 64\}$. Each candidate is timed repeatedly using the same reproducible point subsets at $n = 1000, 2000$, and 5000 . The implementation uses the median runtime across repetitions and then compares the candidate-level summary statistics. The selected threshold is $k = 8$.

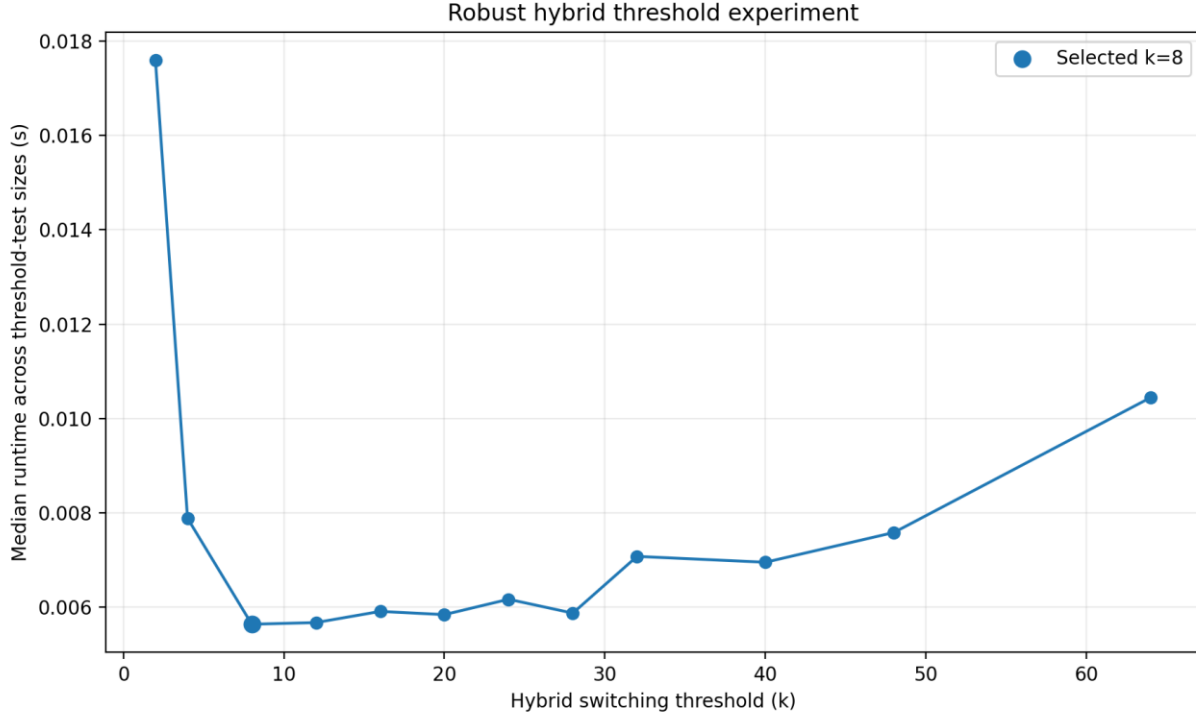


Figure 5. Robust hybrid threshold experiment. The minimum observed candidate is $k = 8$.

8. IMPLEMENTATION AND MODERN TOOL USAGE

8.1 Programming Environment

The solution is implemented in Python using the standard library. The program uses `csv` for dataset parsing, `dataclasses` for point/result representation, `time.perf_counter` for timing, `statistics` for median/mean summaries, and `tracemalloc` for traced peak-memory measurement. Randomized subset selection is controlled by a fixed seed so that the three algorithms receive identical input samples at each benchmark size.

8.2 Data Structures

- **Point:** stores x-coordinate, y-coordinate, and location name.
- **Result:** stores the closest pair, distance, and distance-computation count.
- **DistanceCounter:** explicitly counts every distance evaluation.
- **Px and Py:** x-sorted and y-sorted point views required by the divide-and-conquer method.

8.3 Representative Source-Code Evidence

Brute-Force closest-pair implementation

```
52     def distance_sq(self, a: Point, b: Point) -> float:
53         self.count += 1
54         dx = a.x - b.x
55         dy = a.y - b.y
56         return dx * dx + dy * dy
57
58
59 def brute_force(points: list[Point], counter: DistanceCounter | None = None) -> Result:
60     counter = counter or DistanceCounter()
61     if len(points) < 2:
62         return Result(None, None, math.inf, counter.count)
63
64     best_d2 = math.inf
65     best_pair: tuple[Point | None, Point | None] = (None, None)
66     for i in range(len(points) - 1):
67         for j in range(i + 1, len(points)):
68             d2 = counter.distance_sq(points[i], points[j])
69             if d2 < best_d2:
70                 best_d2 = d2
71                 best_pair = (points[i], points[j])
72
73     return Result(best_pair[0], best_pair[1], math.sqrt(best_d2), counter.count)
74
75
76 def _better(a: Result, b: Result) -> Result:
77     return a if a.distance <= b.distance else b
78
```

Figure 7. Representative source-code evidence: Brute-Force implementation.

Divide-and-Conquer recursive implementation

```

87         else:
88             right_y.append(p)
89         return left_y, right_y
90
91
92 def _closest_recursive(
93     px: list[Point],
94     py: list[Point],
95     counter: DistanceCounter,
96     threshold: int | None = None,
97 ) -> Result:
98     n = len(px)
99     if n <= 3 or (threshold is not None and n <= threshold):
100         return brute_force(px, counter)
101
102     mid = n // 2
103     left_x = px[:mid]
104     right_x = px[mid:]
105     mid_x = px[mid].x
106     left_y, right_y = _split_y(py, left_x)
107
108     left_result = _closest_recursive(left_x, left_y, counter, threshold)
109     right_result = _closest_recursive(right_x, right_y, counter, threshold)
110     best = _better(left_result, right_result)
111     delta2 = best.distance * best.distance
112
113     strip = [p for p in py if (p.x - mid_x) ** 2 < delta2]
114     for i in range(len(strip)):
115         j = i + 1
116         while j < len(strip) and (strip[j].y - strip[i].y) ** 2 < delta2:
117             d2 = counter.distance_sq(strip[i], strip[j])
118             if d2 < delta2:
119                 delta2 = d2
120                 best = Result(strip[i], strip[j], math.sqrt(d2), counter.count)
121             j += 1
122
123     best.distance_computations = counter.count
124     return best
125
126
127 def _prepared(points: list[Point]) -> tuple[list[Point], list[Point]]:
128     px = sorted(points, key=lambda p: (p.x, p.y, p.name))
129     py = sorted(points, key=lambda p: (p.y, p.x, p.name))
130     return px, py
131
132
133 def divide_and_conquer(points: list[Point], counter: DistanceCounter | None = None) -> Result:
134     counter = counter or DistanceCounter()
135     if len(points) < 2:
136         return Result(None, None, math.inf, counter.count)
137     px, py = _prepared(points)
138     result = _closest_recursive(px, py, counter)

```

Figure 8. Representative source-code evidence: Divide-and-Conquer implementation.

OpenFlights data loading and validation

```
140     return result
141
142
143 def hybrid_closest_pair(
144     points: list[Point], threshold: int = 16, counter: DistanceCounter | None = None
145 ) -> Result:
146     if threshold < 2:
147         raise ValueError("threshold must be >= 2")
148     counter = counter or DistanceCounter()
149     if len(points) < 2:
150         return Result(None, None, math.inf, counter.count)
151     px, py = _prepared(points)
152     result = _closest_recursive(px, py, counter, threshold)
153     result.distance_computations = counter.count
154     return result
155
156
157 def load_openflights_airports(path: str | Path) -> list[Point]:
158     points: list[Point] = []
159     with open(path, "r", encoding="utf-8", newline="") as handle:
160         reader = csv.reader(handle)
161         for row in reader:
162             if len(row) < 8:
163                 continue
164             try:
```

Figure 9. Representative source-code evidence: OpenFlights dataset preparation and validation.

Robust threshold search and experimental setup

```
255         "distance_computations": counts[0],
256         "distance": result.distance,
257         "pair_1": result.p1.name if result.p1 else "",
258         "pair_2": result.p2.name if result.p2 else "",
259     }
260
261
262 def threshold_search(
263     points_by_size: dict[int, list[Point]],
264     thresholds: Iterable[int],
265     repeats: int,
266 ) -> tuple[list[dict[str, object]], int]:
267     """Select a robust threshold using median runtime over multiple input sizes."""
268     rows: list[dict[str, object]] = []
269     for threshold in thresholds:
270         per_size_medians: list[float] = []
271         total_comparisons = 0
272         for size in sorted(points_by_size):
273             points = points_by_size[size]
274             times: list[float] = []
275             counts: list[int] = []
276             for _ in range(repeats):
277                 _, elapsed, count = _timed_call(points, "Hybrid", threshold)
278                 times.append(elapsed)
279                 counts.append(count)
280             per_size_medians.append(statistics.median(times))
281             total_comparisons += statistics.median(counts)
282         rows.append(
283             {
284                 "threshold": threshold,
285                 "sizes_tested": ";".join(str(s) for s in sorted(points_by_size)),
286                 "median_time_seconds_across_sizes": statistics.median(per_size_medians),
287                 "mean_time_seconds_across_sizes": statistics.mean(per_size_medians),
288                 "median_distance_computations": total_comparisons / len(points_by_size),
289             }
290         )
291
292     best = min(
293         rows,
294         key=lambda row: (
295             float(row["median_time_seconds_across_sizes"]),
296             int(row["threshold"]),
297         ),
298     )
299     return rows, int(best["threshold"])
300
301
302 def save_csv(path: Path, rows: list[dict[str, object]]) -> None:
303     path.parent.mkdir(parents=True, exist_ok=True)
304     if not rows:
305         return
306     with path.open("w", encoding="utf-8", newline="") as handle:
307         writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
308         writer.writeheader()
309         writer.writerows(rows)
310
311
312 def make_subsets(all_points: list[Point], sizes: list[int], seed: int) -> dict[int, list[Point]]:
313     rng = Random(seed)
314     shuffled = list(all_points)
315     rng.shuffle(shuffled)
316     return {size: shuffled[:size] for size in sorted(set(sizes)) if size <= len(shuffled)}
317
318
319 def write_environment(results_dir: Path, dataset_label: str, point_count: int) -> None:
```

Figure 10. Representative source-code evidence: robust threshold-search and benchmark configuration.

8.4 Reproducibility

The benchmark uses a fixed random seed, identical subsets across algorithms, a fixed threshold after threshold search, repeated timing, and CSV output files. This structure allows the experiment to be rerun with the same dataset and software configuration and provides machine-readable evidence for the report.

9. EXPERIMENTAL METHODOLOGY

9.1 Experimental Configuration

Parameter	Configuration
Dataset	OpenFlights airports.dat
Valid coordinate records	7,698
Largest benchmark	7,500 points
Benchmark sizes	100, 250, 500, 1000, 2000, 5000, 7500
Threshold candidates	2, 4, 8, 12, 16, 20, 24, 28, 32, 40, 48, 64
Threshold-test sizes	1000, 2000, 5000
Timing repetitions	5 per measured configuration in the improved benchmark
Selection criterion	Lowest median runtime across tested sizes
Distance model	Euclidean
Correctness criterion	Same pair and same distance across all three methods
Memory metric	Peak traced allocation via tracemalloc

9.2 Fair Comparison Controls

- All three algorithms are executed against identical point subsets for a given n .
- The point subsets are created once using a reproducible seed.
- Timing and memory measurement are separated so tracemalloc overhead does not contaminate primary runtime measurements.
- Correctness is checked before performance results are accepted.
- The same implementation language and computational environment are used for all approaches.

9.3 Validation

For each benchmark size, the program executes all three algorithms and compares both the minimum distance and the identity of the returned pair. The final run

reported correctness PASS across the tested sizes. At $n = 7500$, every approach identified Shute Harbour Airport and Whitsunday Island Airport as the closest pair with distance 0.001242359449 in the coordinate-space Euclidean model.

10. RESULTS AND VALIDATION

10.1 Final Threshold Result

The robust threshold experiment selected $k = 8$. The threshold plot shows a substantial reduction in median runtime from $k = 2$ through $k = 8$, after which performance begins to fluctuate or degrade as increasingly large base cases cause more direct pair comparisons. The result supports $k = 8$ as the best empirical choice among the tested candidates for this implementation and dataset.

10.2 Execution-Time Scalability

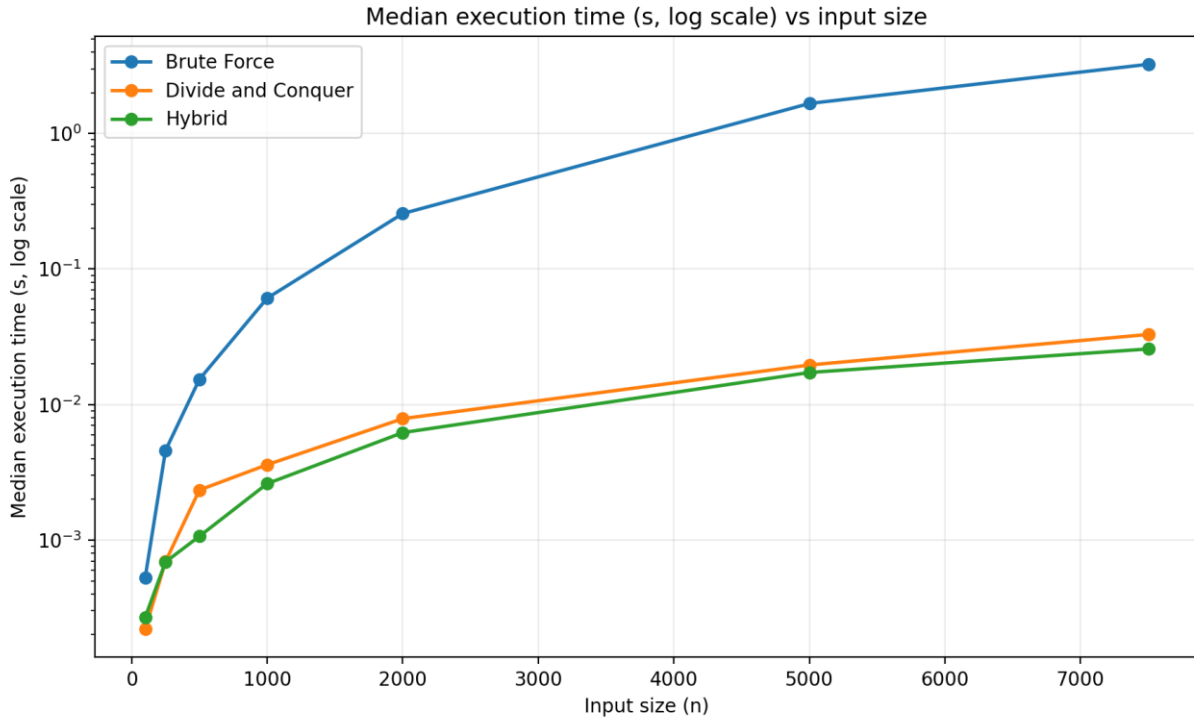


Figure 2. Median execution time (seconds, logarithmic scale) versus input size.

The logarithmic plot makes the asymptotic distinction particularly visible. Brute Force rises sharply as input size increases, while Divide and Conquer and Hybrid remain in the much lower time range. The Hybrid curve is consistently competitive and becomes clearly lower than Divide and Conquer at the larger tested input sizes.

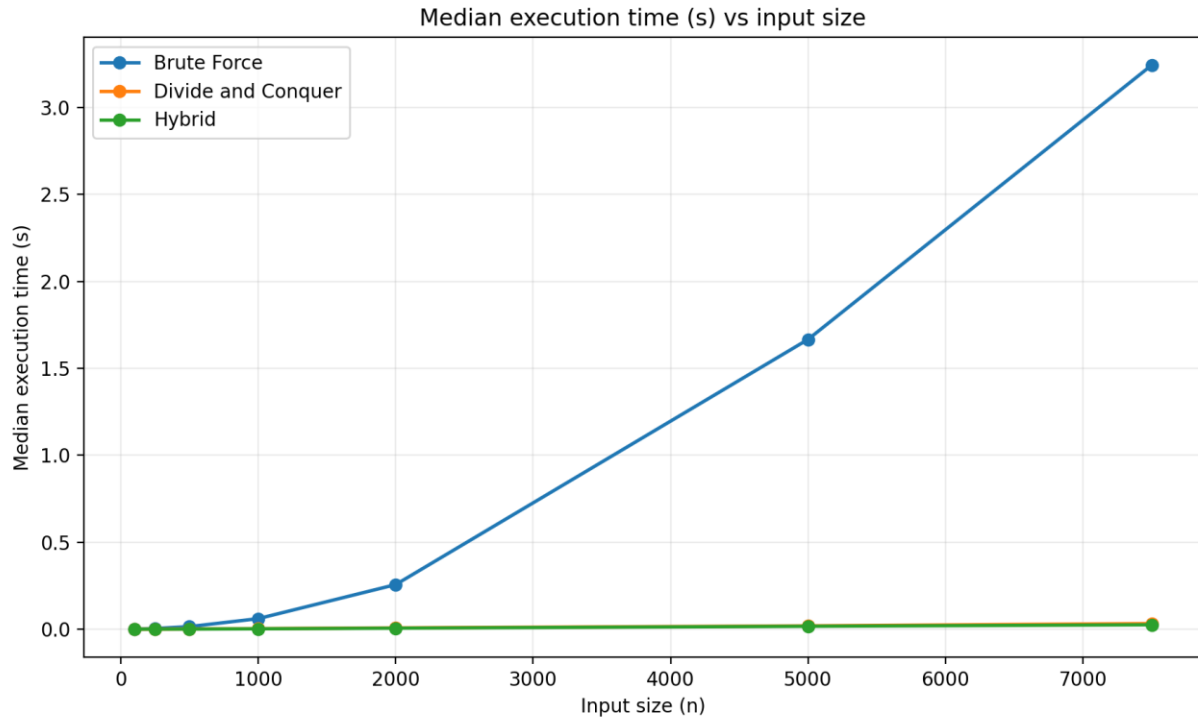


Figure 6. Median execution time (seconds) versus input size on a linear scale.

The linear-scale view reinforces the same conclusion: the Brute-Force curve dominates the vertical range at larger n , while the Divide-and-Conquer and Hybrid curves remain comparatively flat at the scale of the experiment. This difference is consistent with the theoretical $O(n^2)$ versus $O(n \log n)$ growth rates.

10.3 Distance-Calculation Efficiency

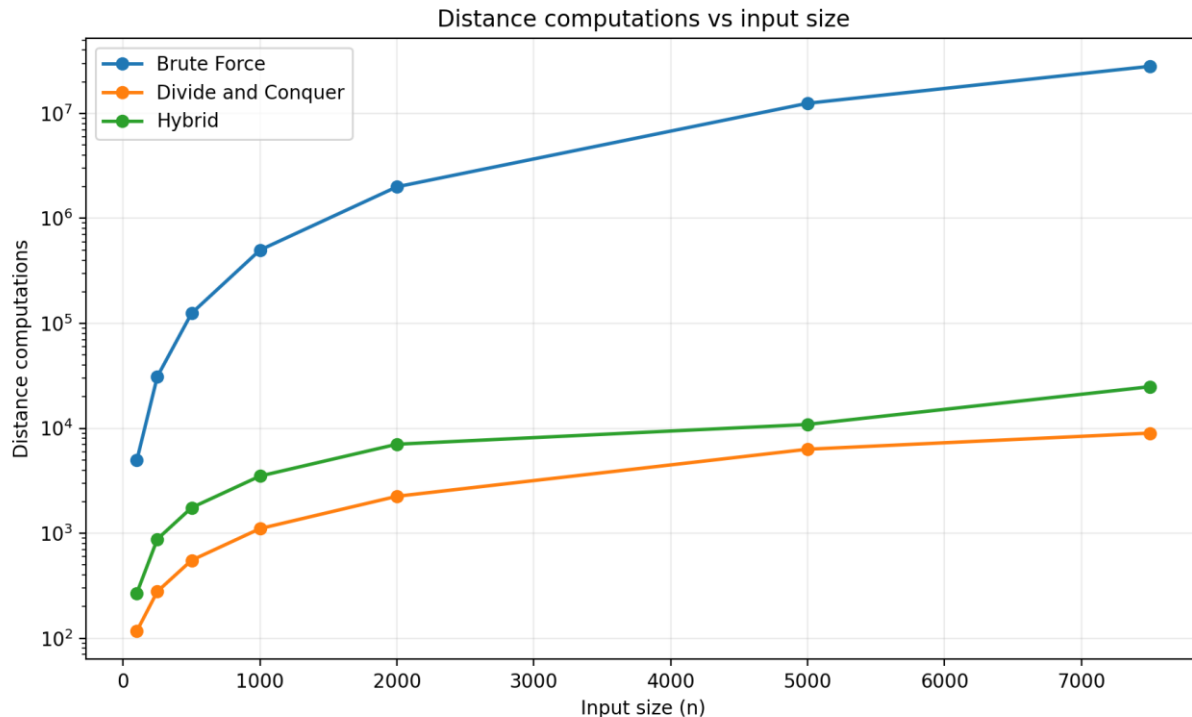


Figure 3. Distance computations versus input size.

The distance-count curve directly demonstrates why Brute Force becomes expensive. At $n = 7500$ it performs 28,121,250 distance calculations. Divide and Conquer performs only 9,002, while Hybrid performs 24,887. The Hybrid method uses more distance calculations than pure Divide and Conquer in this implementation, but still obtains a lower execution time, indicating that reduced recursive/base-case overhead is beneficial enough to outweigh the additional local comparisons.

10.4 Memory Behavior

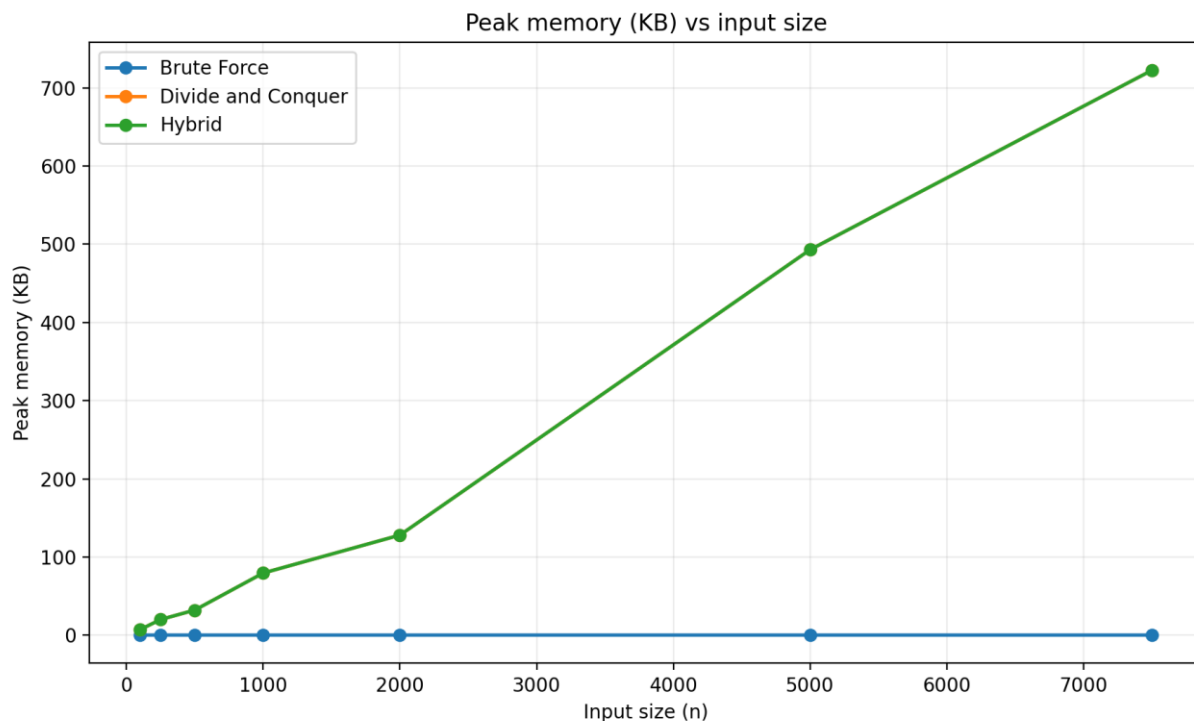


Figure 4. Peak traced memory allocation versus input size.

The memory chart shows the expected distinction between the algorithms' auxiliary allocation behavior. Brute Force remains close to zero additional traced allocation, whereas Divide and Conquer and Hybrid increase with input size because they construct sorted lists, recursive subproblem structures, and strip-related data. At $n = 7500$, both Divide and Conquer and Hybrid report 722.5 KB of peak traced allocation. These figures should be interpreted as measured additional allocations observed by tracemalloc during the algorithm call, not as total application RAM consumption.

Algorithm	Median time (s)	Distance calculations	Peak traced memory (KB)	Correct closest pair
Brute Force	3.242633	28,121,250	0.2	Shute Harbour ↔ Whitsunday Island
Divide and Conquer	0.032881	9,002	722.5	Shute Harbour ↔ Whitsunday Island
Hybrid (k=8)	0.025702	24,887	722.5	Shute Harbour ↔ Whitsunday Island

Table 5. Final performance at the largest benchmark size ($n = 7500$); all methods report distance 0.001242359449.

10.5 Derived Performance Indicators

Indicator	Value	Interpretation
Hybrid speed improvement vs Divide and Conquer	21.83%	Hybrid median runtime is lower at $n = 7500$.
Hybrid speed improvement vs Brute Force	99.21%	Hybrid avoids almost all of the quadratic runtime cost at the largest size.
Brute Force distance-count reduction achieved by Hybrid	99.91%	Hybrid uses 24,887 versus 28,121,250 distance calculations.
Divide-and-Conquer distance-count advantage	9,002 comparisons	Pure Divide and Conquer performs the fewest measured distance calculations at $n = 7500$.
Correctness	PASS	All three methods return the same closest pair and distance.

Table 6. Derived performance indicators at $n = 7500$.

10.6 Correctness Validation

Correctness is not inferred from runtime. The implementation explicitly executes all three algorithms on the same points and checks the pair identity and distance using a strict absolute tolerance of $1e-10$. The final experiment reports a common closest pair: Shute Harbour Airport and Whitsunday Island Airport, with a Euclidean coordinate-space distance of 0.001242359449.

11. ANALYSIS AND ENGINEERING DISCUSSION

11.1 Brute Force

Brute Force is simple, transparent, and has minimal auxiliary allocation. However, its quadratic pair enumeration becomes the dominant cost as n grows. At $n = 7500$, 28,121,250 distance calculations are required. The 3.242633-second median runtime is roughly two orders of magnitude larger than the two scalable approaches.

11.2 Divide and Conquer

Divide and Conquer provides the expected scalable behavior. Its 0.032881-second median at $n = 7500$ is approximately 99.0% faster than Brute Force. The method also achieves the smallest measured distance-calculation count, 9,002. Its main practical cost in this Python implementation is the auxiliary data handling required for recursive splitting and sorted views.

11.3 Hybrid

The Hybrid method provides the strongest measured execution time at the largest tested size. With $k = 8$, it reaches 0.025702 seconds, which is 21.83% faster than pure Divide and Conquer and 99.21% faster than Brute Force. It performs more distance calculations than pure Divide and Conquer, showing that distance-count minimization and wall-clock minimization are not identical objectives. The experiment therefore demonstrates why a practical algorithm should be optimized using multiple metrics rather than a single operation count.

11.4 Threshold Effect

Very small thresholds force many recursive levels and increase framework overhead. Very large thresholds perform more direct pair comparisons within recursive subproblems. The observed minimum at $k = 8$ represents a balance between those effects for the tested dataset and Python implementation. Because the threshold is an implementation-dependent constant factor, it should be revalidated when hardware, language runtime, data distribution, or code structure changes.

11.5 Theory Versus Experiment

The empirical results align with the theoretical analysis. The Brute-Force method exhibits the steep growth expected from $\Theta(n^2)$, while Divide and Conquer and Hybrid scale much more gently in the tested range. The exact timing curves do not form perfect mathematical functions because real execution includes interpreter overhead, memory allocation, CPU scheduling, and data-dependent branch behavior. Nevertheless, the broad shape of the curves is consistent with the asymptotic expectations.

12. ENGINEERING DECISION

12.1 Selected Approach

The recommended approach for the case study is the Adaptive Hybrid Closest-Pair algorithm with switching threshold $k = 8$.

12.2 Quantitative Justification

- At $n = 7500$, Hybrid reduces median runtime by 21.83% relative to Divide and Conquer.
- At $n = 7500$, Hybrid reduces median runtime by 99.21% relative to Brute Force.
- Hybrid and the other methods return exactly the same closest pair and distance.

- Hybrid retains the $\Theta(n \log n)$ asymptotic framework of Divide and Conquer while reducing practical recursive overhead through small-subproblem Brute-Force evaluation.
- The threshold is selected experimentally rather than assumed, directly satisfying the optimization objective of the assignment.

12.3 Trade-Offs

Approach	Strengths	Limitations	Best use
Brute Force	Simple, easy to verify, low auxiliary allocation	$\Theta(n^2)$ and quickly becomes expensive	Very small datasets or correctness baseline
Divide and Conquer	$\Theta(n \log n)$, few distance calculations, strong scalability	Higher implementation complexity and auxiliary allocation	Large datasets where general-purpose scalability is required
Hybrid	Best measured runtime in this study; combines both methods; tunable	Threshold is implementation/data dependent; more local comparisons than pure D&C	Large datasets with repeated workloads and a calibrated runtime environment

12.4 Final Engineering Statement

For the studied OpenFlights workload, the Hybrid method is the most suitable engineering choice because it preserves the scalability profile of Divide and Conquer while delivering the lowest measured median runtime at the largest benchmark size. The result should be understood as an empirical optimization for the tested software environment rather than as a universal claim that $k = 8$ is optimal for every implementation or dataset.

13. BROADER CONSIDERATIONS AND SDG 9

The assignment maps the work to SDG 9 – Industry, Innovation and Infrastructure. [1] The project contributes to this theme by demonstrating how algorithmic optimization can reduce computational time and make large-scale spatial analysis more practical.

- Efficient utilization of computational resources through reduced runtime.
- Reduction of unnecessary pairwise distance calculations compared with Brute Force.
- Improved scalability as the number of spatial records grows.
- A practical foundation for location-based infrastructure analysis, facility screening, and service-coverage studies.

- An optimization methodology that can be adapted to other spatial datasets and computing environments.

In a production system, the same algorithmic idea could support pre-screening or proximity analysis before more sophisticated geographic models are applied. The present case study intentionally remains within the assignment's Euclidean model rather than making geographic-distance claims beyond what the data representation supports.

14. LIMITATIONS

1. The experiment treats latitude and longitude as planar coordinates because the assignment explicitly requires Euclidean distance. This means the reported distance is a coordinate-space Euclidean value, not a geodesic distance on Earth.
2. The measured optimal threshold $k = 8$ is specific to this implementation, dataset, Python runtime, and test environment. A different environment may produce a different optimum.
3. Peak memory values are traced allocations during algorithm calls rather than total system memory consumption.
4. The benchmark stops at $n = 7500$ because the quadratic Brute-Force approach becomes increasingly expensive; the full dataset still contains 7,698 valid records.
5. Timing measurements can vary slightly due to operating-system scheduling, background tasks, CPU frequency changes, and runtime effects.

15. FUTURE OPTIMIZATION

- Calibrate the threshold automatically on the target deployment machine before production execution.
- Use geodesic distance or projected coordinate systems when geographic accuracy rather than assignment-level Euclidean distance is required.
- Investigate iterative implementations and lower-allocation data structures to reduce Python object and list overhead.
- Extend the benchmark to additional real-world facility datasets such as weather stations or cellular infrastructure locations.
- Profile the implementation to separate sorting, recursion, strip construction, and distance computation costs.
- Explore parallel or vectorized strategies for very large datasets where the hardware and workload justify additional complexity.

16. CONCLUSION

This case study successfully addressed the Closest-Pair-of-Points problem using a real-world geospatial dataset and three algorithmic strategies. Brute Force provides a useful correctness baseline but exhibits $\Theta(n^2)$ growth and becomes expensive as input size increases. Divide and Conquer reduces the complexity to $\Theta(n \log n)$ through recursive partitioning and strip processing. The developed Hybrid approach extends this design by switching to Brute Force for small subproblems.

The experimental threshold search selected $k = 8$. At the largest benchmark size of $n = 7500$, the Hybrid algorithm achieved a median runtime of 0.025702 seconds, compared with 0.032881 seconds for Divide and Conquer and 3.242633 seconds for Brute Force. All three methods identified the same closest pair, Shute Harbour Airport and Whitsunday Island Airport, with distance 0.001242359449 in the specified coordinate-space Euclidean model. These results support the engineering decision to use the Hybrid approach for this case study.

The central DAA lesson is that asymptotic analysis explains scalability, but practical algorithm engineering also depends on constant factors, data structures, implementation overhead, and experimentally calibrated design parameters. The Hybrid approach demonstrates how those factors can be exploited without changing the underlying asymptotic class.

17. INDIVIDUAL STUDENT REFLECTIONS

17.1 BANDI MOHITH MANOJ – Hybrid Development and Optimization

The main learning from this project was that a hybrid algorithm is not created simply by placing two existing algorithms in the same program. The important design task is deciding when each method should be used. I learned how the threshold changes practical runtime even when the asymptotic complexity remains $\Theta(n \log n)$. The main challenge was avoiding an arbitrary threshold. Repeated measurements across multiple input sizes helped convert the threshold choice into an evidence-based engineering decision. If more resources were available, I would calibrate the threshold automatically for different hardware and investigate lower-allocation data structures. The project also showed how algorithmic optimization can contribute to SDG 9 by improving the computational efficiency of infrastructure-oriented spatial analysis.

17.2 KODANDA RAM T – Dataset and Experimental Evaluation

My major learning was that experimental design is as important as algorithm implementation. The same algorithms must receive identical data and must be measured under identical conditions for a comparison to be meaningful. Preparing

the OpenFlights coordinates, validating records, and measuring distance calculations and memory helped connect textbook complexity to real execution behavior. The challenge was that timing can vary, so repeated measurements and median values provide a more stable basis for comparison. With additional time, I would extend the evaluation to more datasets and more benchmark repetitions. This work demonstrates that efficient algorithms can support more scalable data processing for infrastructure applications.

17.3 M YASODHA KRISHNA – Analysis, Validation and Documentation

This project strengthened my understanding of the relationship between theoretical analysis and measured performance. Working through the recurrence $T(n) = 2T(n/2) + O(n)$ and the Master Theorem made the $\Theta(n \log n)$ result concrete, while correctness validation showed why experimental output must also be checked. A major challenge was explaining why the Hybrid algorithm can be faster even when it performs more distance calculations than pure Divide and Conquer. The answer is that computational cost includes more than the number of distance operations; recursive and data-management overhead also matter. With more resources, I would add a richer statistical analysis and compare the approach with geodesic distance models. The project connects algorithmic efficiency with the broader goal of building scalable computing foundations for intelligent infrastructure.

18. REFERENCES

- [1] Dr. F. Mary Harin Fernandez, “CSA0619 Design and Analysis of Algorithms – Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing,” Course Assignment Brief, SIMATS, Academic Year 2026–2027.
- [2] OpenFlights, “Airport, airline and route data – Airport database,” OpenFlights. Available: <https://openflights.org/data>
- [3] M. I. Shamos and D. Hoey, “Closest-Point Problems,” Proceedings of the 16th Annual IEEE Symposium on Foundations of Computer Science, pp. 151–162, 1975. DOI: 10.1109/SFCS.1975.8.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.
- [5] Python Software Foundation, “time — Time access and conversions,” Python Documentation. Available: <https://docs.python.org/3/library/time.html>
- [6] Python Software Foundation, “tracemalloc — Trace memory allocations,” Python Documentation. Available: <https://docs.python.org/3/library/tracemalloc.html>

[7] Python Software Foundation, “csv — CSV File Reading and Writing,” Python Documentation. Available: <https://docs.python.org/3/library/csv.html>

19. ASSIGNMENT-REQUIREMENT COVERAGE MATRIX

The following matrix makes the report traceable to the faculty assignment and its assessment rubric.

Assignment requirement	Report evidence
Problem understanding and formulation	Sections 1 and 2: case background, inputs, outputs, assumptions, constraints
Real-world geospatial dataset	Section 3: OpenFlights description, fields, preprocessing and record count
Brute-Force implementation	Sections 5, 6 and 7; source-code evidence in Section 8
Divide-and-Conquer implementation	Sections 5, 6 and 7; source-code evidence in Section 8
Developed Hybrid algorithm	Sections 7 and 12
Adaptive threshold switching	Sections 7 and 10; threshold figure
Experimental threshold determination	Section 7.6 and Figure 5; selected $k = 8$
Big-O, Ω , Θ analysis	Section 6
Master Theorem recurrence	Section 6.3
Execution-time comparison	Section 10 and Figures 2 and 6
Distance-computation comparison	Section 10 and Figure 3
Memory comparison	Section 10 and Figure 4
Correctness validation	Section 10.6
Scalability analysis	Sections 10 and 11
Engineering decision	Section 12
SDG 9 relevance	Section 13
Limitations and future optimization	Sections 14 and 15
Individual reflection	Section 17
References and AI/tool acknowledgement	Section 18

Table 7. Assignment-requirement coverage matrix.

APPENDIX A – FINAL EXPERIMENTAL SUMMARY

Metric	Final value
Dataset	OpenFlights airports.dat
Valid coordinate records	7,698

Metric	Final value
Selected hybrid threshold	8
Largest benchmark size	7,500
Brute Force	3.242633 s; 28,121,250 distance calculations; 0.2 KB
Divide and Conquer	0.032881 s; 9,002 distance calculations; 722.5 KB
Hybrid	0.025702 s; 24,887 distance calculations; 722.5 KB
Common closest pair	Shute Harbour Airport and Whitsunday Island Airport
Common Euclidean distance	0.001242359449

APPENDIX B – REPOSITORY STRUCTURE

- Dataset/airports.dat – real-world coordinate dataset.
- Results/performance_results.csv – benchmark measurements.
- Results/threshold_results.csv – threshold-search measurements.
- Results/selected_threshold.txt – selected k value.
- Results/environment.txt – execution environment.
- Results/final_summary.txt – report-ready summary.
- Results/graphs/ – generated visualizations.
- Source_Code/closest_pair_hybrid.py – main implementation.
- Source_Code/plot_results.py – graph generation.
- Report/ – final documentation.

APPENDIX C – AI-ASSISTED TOOL USE

AI-assisted tools were used during drafting and development support for code organization, report structuring, interpretation of the measured results, and formatting. The experimental values reported in this document are taken from the executed project output supplied for the final run; they are not invented or substituted with illustrative data.